

Reviewer Pack — Additive Energy of Fourier Coefficients Bounds ACC⁰ Circuit ...

Entry aa6c5017c32c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 09:59:21 UTC

Additive Energy of Fourier Coefficients Bounds ACC⁰ Circuit Size for Sipser Function

Entry ID: aa6c5017c32c Verdict: INCONCLUSIVE Recorded: 2026-05-20 09:59:21 UTC

1. Conjecture

Field A (mathematical branch): Additive combinatorics **Field B** (complexity object): ACC⁰ circuit complexity

Statement:

For the Sipser function on n variables, the additive energy of its Fourier coefficients is $\Omega(2^{\{n/2\}})$, implying that any ACC⁰ circuit computing it requires $\Omega(2^{\{n/2\}})$ gates.

Rationale (proposer’s reasoning):

Additive energy quantifies the number of additive configurations in a set, reflecting structural complexity. If the Sipser function’s Fourier coefficients exhibit high additive energy, ACC⁰ circuits—limited in their ability to handle structured additive dependencies—would struggle to compute them efficiently.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 15e92e80a1518dbf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def fast_walsh_hadamard_transform(x):
        n = len(x)
        if n == 1:
            return x
        even = fast_walsh_hadamard_transform(x[0::2])
        odd = fast_walsh_hadamard_transform(x[1::2])
        result = [0] * n
        for i in range(n // 2):
            result[i] = even[i] + odd[i]
            result[i + n // 2] = even[i] - odd[i]
        return result

    def additive_energy(coeffs):
        n = len(coeffs)
        energy = 0
        for i in range(n):
            for j in range(i + 1, n):
                if coeffs[i] * coeffs[j] != 0:
                    energy += 1
        return energy

    def sipser_function(n, x):
        result = 0
        for i in range(n):
            result ^= x[i]
        return result

    n = random.randint(5, 40)
    x = [random.randint(0, 1) for _ in range(n)]
    coeffs = fast_walsh_hadamard_transform([sipser_function(n, x[:i] + (x[i] ^ 1,) + x[i+1:]) for
    energy = additive_energy(coeffs)

    return {
        "metric_name": "additive_energy",
```

```

        "metric_value": energy,
        "instances_tested": 1,
        "conjecture_holds": energy >= 2 ** (n / 2),
        "counterexample": "" if energy >= 2 ** (n / 2) else f"Graph with n={n}, coeffs={coeffs}"
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.randint(1, 1000000) for _ in range(30)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_energy = sum(res["metric_value"] for res in results) / len(results)
    std_energy = math.sqrt(sum((res["metric_value"] - mean_energy) ** 2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_energy} std={std_energy} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ff01815f.py", line 66, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ff01815f.py", line 50, in run_trial
    coeffs = fast_walsh_hadamard_transform([sipser_function(n, x[:i] + (x[i] ^ 1,) + x[i+1:]) for i in range(n-1)])
                                         ~~~~~^~~~~~

```

TypeError: can only concatenate list (not "tuple") to list

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to type error before producing data, preventing evaluation of support fraction or counterexamples | next: Fix the tuple-list concatenation error in the fast_walsh_hadamard_transform call

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	138836
2	propose	ollama_remote	qwen3:8b	0	0	132298
3	preregistration	ollama_remote	qwen3:8b	0	0	31666
4	novelty	ollama_remote	qwen3:8b	0	0	27328
5	novelty	ollama_remote	qwen3:8b	0	0	22059
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13106
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8200
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8032
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7942
10	judge	ollama_remote	qwen3:8b	0	0	51999

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 441466 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/aa6c5017c32c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/aa6c5017c32c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/aa6c5017c32c.tar.gz` (if generated)